

Proof Report for RefJasminNTT.ec

Codex Proof Report

28 April 2026

Abstract

This report documents what was proved in `RefJasminNTT.ec` and how the proof is structured. The file establishes direct loop-level pRHL refinements from the extracted Jasmin NTT cores in `Hpoly_extract.M` to the imperative EasyCrypt reference procedures in `NTT_Fq.NTT`. The forward theorem proves that `Hpoly_extract.M._poly_ntt` refines `NTT_Fq.NTT.ntt`. The inverse theorem proves that `Hpoly_extract.M._poly_invntt` refines `NTT_Fq.NTT.invntt`, with the representation relation strengthened at the postcondition to account for the final Montgomery-encoded scaling performed by the Jasmin routine. The proof is not an algebraic proof of the NTT against `Rq`; it is the concrete extraction-to-imperative-specification layer.

1 Scope of the File

`RefJasminNTT.ec` is deliberately narrower than the algebraic correctness development. Its header states the intended boundary: it targets `extract/Hpoly_extract.ec` and stops at refinement to the imperative EasyCrypt NTT procedures in `NTT_Fq.ec`. Thus the proof chain covered here is

$$\text{extracted Jasmin core} \implies \text{imperative EasyCrypt NTT model.}$$

The later mathematical step

$$\text{imperative EasyCrypt NTT model} \implies \text{abstract ring-theoretic NTT}$$

is outside the scope of this file.

The two central theorems are:

```
poly_ntt_core_ref,  
poly_invntt_core_ref.
```

Two wrapper equivalences are also proved:

$$\text{poly_ntt_jazz_ref} \quad \text{and} \quad \text{poly_invntt_jazz_ref},$$

showing that the public extracted wrappers are just inlined calls to the core procedures. These wrapper proofs are short `proc; inline*; sim` arguments; the substantial work is in the core refinements.

2 Target Procedures

The right-hand programs are the imperative specifications in `NTT_Fq.NTT`. The forward procedure `NTT.ntt` uses the standard descending layer lengths

$$128, 64, 32, 16, 8, 4, 2, 1$$

and, for each butterfly, computes

$$t = \zeta \cdot r_{j+\ell}, \quad r'_{j+\ell} = r_j - t, \quad r'_j = r_j + t.$$

The inverse procedure `NTT.invntt` uses ascending layer lengths

$$1, 2, 4, 8, 16, 32, 64, 128$$

and computes

$$t = r_j, \quad r'_j = t + r_{j+\ell}, \quad r'_{j+\ell} = \zeta \cdot (t - r_{j+\ell}).$$

After the inverse butterfly layers, `NTT.invntt` performs a final scaling loop:

$$r'_j = r_j \cdot \text{zetas_inv}[255].$$

The left-hand programs are the corresponding extracted Jasmin cores:

$$\text{Hpoly_extract.M._poly_ntt} \quad \text{and} \quad \text{Hpoly_extract.M._poly_invntt}.$$

These programs manipulate `BArray1024.t` buffers containing 256 signed 32-bit coefficients and call the extracted Montgomery multiplication routine `Hpoly_extract.M.\_fqmul`.

3 Representation Relations

3.1 Base Representation

The base representation is inherited from `NTT_Fq.ec`. A machine word is interpreted as a field coefficient by

$$\text{word_to_coeff}(w) = \text{incoeff}(\text{W32.to_sint}(w)).$$

The polynomial readback from a 1024-byte buffer is

$$\text{barray256_to_poly}(rp)[i] = \text{word_to_coeff}(\text{BArray1024.get32 } rp \ i).$$

The relation

$$\text{poly_repr}(rp, p)$$

states that this readback equals the coefficient array p .

The bounded relation used by the final theorems is

$$\text{poly_repr_bound}(rp, p, s) \equiv \text{poly_repr}(rp, p) \wedge \forall i \in \{0, \dots, 255\}. \text{Fq.bw32}(\text{BArray1024.get32 } rp \ i, s).$$

Here `Fq.bw32 w s` records that the signed value of w is representable within the signed bound 2^s . This is the central device used to rule out machine-word overflow in additions, subtractions, and products supplied to `__fqmul`.

3.2 Pointwise Bounds

`RefJasminNTT.ec` introduces a more flexible pointwise bounded relation:

$$\text{barray256_bound_by}(rp, bd) \equiv \forall i \in \{0, \dots, 255\}. \text{Fq.bw32}(\text{BArray1024.get32 } rp \ i, \ bd(i)).$$

and

$$\text{poly_repr_bound_by}(rp, p, bd) \equiv \text{poly_repr}(rp, p) \wedge \text{barray256_bound_by}(rp, bd).$$

The pointwise form is necessary because, inside a butterfly loop, some coefficients in the current stage have already been updated and therefore have a larger bound than coefficients still waiting to be processed.

The following structural lemmas make this relation usable in loop invariants:

- `barray256_bound_by_get`: extracts the bound for a concrete index;
- `barray256_bound_by_weaken`: weakens a pointwise bound;
- `barray256_bound_by_set32`: preserves a bound after one store;
- `barray256_bound_by_set32_change`: changes the pointwise bound after one store while weakening all untouched entries;
- `barray256_bound_by_const` and `barray256_bound_by_const_bound`: convert between uniform bounds and pointwise constant bounds.

4 Montgomery Table Bridges

The extracted Jasmin multiplication is Montgomery multiplication. Consequently, twiddle constants loaded from `jzetas` and `jzetas_inv` are Montgomery encoded. `RefJasminNTT.ec` imports the concrete table facts from `NTT_Fq.ec` and packages them as cancellation lemmas.

For the forward table, the key bridge is

$$\text{jzetas_get_cancel} : \text{word_to_coeff}(\text{jzetas}[i]) \cdot R^{-1} = \text{zetas}[i] \quad (1 \leq i < 256).$$

For the inverse table, the analogous bridge is

$$\text{jzetas_inv_get_cancel} : \text{word_to_coeff}(\text{jzetas_inv}[i]) \cdot R^{-1} = \text{zetas_inv}[i] \quad (0 \leq i < 255).$$

The final inverse scaling entry is special. The table entry at index 255 is used by the Jasmin final loop. Its cancellation lemma is

$$\text{jzetas_inv_255_scale_cancel} : \text{word_to_coeff}(\text{jzetas_inv}[255]) \cdot R^{-1} = \text{scale255} \cdot R.$$

This identity explains why the inverse theorem does not end with `poly_repr_bound res{1} res{2}` 16. The Jasmin final loop produces a Montgomery-scaled image of the right-hand EasyCrypt result.

5 Arithmetic Correctness of Extracted Multiplication

The proof factors every call to `Hpoly_extract.M.__fqmul` through specialized Hoare and probabilistic Hoare lemmas. At the lowest level, `fqmul_word_to_coeff_mul_bound_h` proves that if the signed integer product of the two inputs lies in the range accepted by the signed Montgomery reducer, then the result has both the correct field value and a 16-bit signed bound:

$$\begin{aligned} \text{word_to_coeff}(res) &= \text{incoeff}(a) \cdot \text{incoeff}(b) \cdot R^{-1}, \\ \text{Fq.bw32}(res, 16). \end{aligned}$$

The probabilistic version `fqmul_word_to_coeff_mul_bound_ph` adds the losslessness side condition, using `Fq.fqmul_11`.

The product bound used by all butterfly calls is

$$\text{fqmul_product_bound_16_24}.$$

It states that a 16-bit twiddle multiplied by a 24-bit coefficient fits in the input range required by the Montgomery reduction proof:

$$-\frac{R}{2}q \leq \text{to_sint}(a)\text{to_sint}(b) < \frac{R}{2}q.$$

The forward and inverse loop invariants are designed precisely so that every coefficient supplied to `__fqmul` can be weakened to a 24-bit bound, while every twiddle table entry has a 16-bit bound.

6 Forward Refinement

6.1 Statement

The forward core theorem is:

$$\begin{aligned} \text{equiv poly_ntt_core_ref} : \\ &\text{Hpoly_extract.M._poly_ntt} \sim \text{NTT_Fq.NTT.ntt} : \\ &\text{NTT_Fq.poly_repr_bound } r_{1\backslash} \ r_{2\backslash} \ 16 \ /\backslash \\ &\text{zetas}_{2\backslash} = \text{NTT_Fq.zetas} \\ &\implies \text{NTT_Fq.poly_repr_bound } res_{1\backslash} \ res_{2\backslash} \ 24. \end{aligned}$$

The precondition says that the Jasmin input buffer represents the right-hand coefficient array and all input words are initially 16-bit bounded. The postcondition says that the two procedures return represented arrays, with a 24-bit output bound. The 24-bit bound is not accidental: each forward stage can increase the coefficient bound by one bit, and there are eight butterfly layers.

6.2 Stage Bookkeeping

Forward loop control is summarized by:

$$\text{fwd_len_ok}(len)$$

for the allowed values 128, 64, ..., 1, 0, and

$$\text{fwd_stage_bound}(len)$$

for the uniform stage bound:

len	128	64	32	16	8	4	2	1	0
$fwd_stage_bound(len)$	16	17	18	19	20	21	22	23	24

The table counter is tracked by

$$fwd_zbase(len),$$

whose values correspond to the first twiddle index of the current layer:

$$0, 1, 3, 7, 15, 31, 63, 127, 255.$$

The lemmas `fwd_stage_next`, `fwd_len_next_ok`, and `fwd_zbase_next` prove that these values evolve exactly as the two programs update `len` and `zetasctr`.

6.3 Nested Loop Invariants

The proof uses three nested pRHL **while** invariants matching the three nested loops of the programs.

Outer invariant. At the start of each layer:

$$\begin{aligned} & \text{poly_repr}(rp_1, r_2), \\ & \text{barray256_bound_by}(rp_1, \lambda i. fwd_stage_bound(len_1)), \\ & \text{zetasp}_1 = \text{jzetas}, \quad \text{zetasp}_2 = \text{NTT_Fq.zetas}, \\ & \text{zetasctr}_1 = \text{zetasctr}_2, \quad \text{len}_1 = \text{len}_2, \\ & \text{fwd_len_ok}(len_1), \quad \text{zetasctr}_1 = \text{fwd_zbase}(len_1). \end{aligned}$$

This invariant aligns the two programs at layer boundaries and asserts the uniform bound for the current stage.

Middle invariant. Inside a layer, `start` marks how much of the array has already been processed. The bound is

$$fwd_middle_bound(sz, start, i) = \begin{cases} sz + 1, & 0 \leq i < start, \\ sz, & \text{otherwise.} \end{cases}$$

Thus coefficients in completed blocks have grown by one bit, while the rest of the layer remains at the original stage bound.

Inner invariant. Inside one block, the inner-loop bound is

$$fwd_inner_bound(sz, \ell, start, j, i).$$

It gives bound $sz+1$ to coefficients in completed butterflies and bound sz to the two current butterfly inputs. The lemmas `fwd_inner_current`, `fwd_inner_weaken_unchanged`, and `fwd_inner_exit_to_middle` justify, respectively, reading the current input bounds, preserving untouched entries after a store, and converting the inner bound back to the middle bound when the block ends.

6.4 Butterfly Step

The forward butterfly proof is split into two conceptual pieces.

First, `forward_twiddle_product_bound_call_ph` verifies the call to `__fqmul`. It uses:

- `jzetas_get_cancel` to convert the Montgomery table entry into the abstract ζ ;
- `forward_twiddle_product_bound_from_inner` to prove the machine-word product bound required by `__fqmul`;
- `fqmul_word_to_coeff_mul_bound_ph` to obtain both semantic correctness and a 16-bit result bound.

Second, `forward_butterfly_step_bound_by` verifies the two stores:

$$r_{j+\ell} \leftarrow r_j - t, \quad r_j \leftarrow r_j + t.$$

The field-level equality is obtained through `word_to_coeff_add` and `word_to_coeff_sub`; their preconditions are exactly the pointwise `bw32` bounds carried in the invariant. The output bound is raised from sz to $sz + 1$ on the two updated positions using `forward_butterfly_bounds`.

6.5 Completion of the Forward Proof

At inner-loop exit, the proof derives $j = start + \ell$ and converts `fwd_inner_bound` to `fwd_middle_bound`. At middle-loop exit, it derives $start = 256$, upgrades the whole layer to $sz + 1$, and uses `fwd_stage_next` to re-enter the outer invariant at the next value $\ell/2$. At outer-loop exit, $\ell = 0$, and `fwd_stage_bound(0)` is 24, giving the final postcondition

$$\text{poly_repr_bound}(res_1, res_2, 24).$$

7 Inverse Refinement

7.1 Statement

The inverse core theorem is:

```
equiv poly_invntt_core_ref :
  Hpoly_extract.M._poly_invntt ~ NTT_Fq.NTT.invntt :
  NTT_Fq.poly_repr_bound rp\{1\} r\{2\} 16 /\
  zetas_inv\{2\} = NTT_Fq.zetas_inv
==> NTT_Fq.poly_repr_bound res\{1\}
      (NTT_Fq.array256_mont res\{2\}) 16.
```

The postcondition is intentionally strengthened relative to a plain coefficient representation. The right-hand program multiplies by `zetas_inv[255]` at the abstract coefficient level, whereas the left-hand Jasmin code calls Montgomery multiplication with the concrete table entry `jzetas_inv[255]`. The table bridge proves that this machine operation produces the Montgomery-scaled coefficient

$$r_j \cdot \text{scale255} \cdot R.$$

Therefore the final represented polynomial is

$$\text{array256_mont}(res_2),$$

not res_2 itself.

7.2 Inverse Stage Bookkeeping

The inverse layer control is summarized by:

$$\text{inv_len_ok}(len)$$

for the values $1, 2, \dots, 128, 256$, and

$$\text{inv_stage_bound}(len)$$

with values:

len	1	2	4	8	16	32	64	128	256
$\text{inv_stage_bound}(len)$	16	17	18	19	20	21	22	23	24.

The inverse table counter is described by

$$\text{inv_zbase}(len) = 0, 128, 192, 224, 240, 248, 252, 254, 255.$$

The lemmas `inv_stage_next`, `inv_len_next_ok`, and `inv_stage_zbase_exit` prove that these values match the update $\ell \leftarrow 2\ell$ and the inverse table traversal.

7.3 Inverse Butterfly Layers

The inverse proof reuses the same three-level invariant shape as the forward proof, replacing `fwd_*` operators with `inv_*`. The important semantic difference is the butterfly:

$$r'_j = r_j + r_{j+\ell}, \quad r'_{j+\ell} = \zeta \cdot (r_j - r_{j+\ell}).$$

The extracted code computes the difference in a temporary word and then calls `__fqmul`. The proof therefore needs two additional facts:

- `inverse_twiddle_product_bound_from_inner`: the word difference $rp[j] - rp[j+\ell]$ is bounded by `inv_stage_bound len + 1`;
- `inverse_twiddle_product_call_bound_from_inner`: after weakening this difference to a 24-bit bound and combining it with a 16-bit inverse twiddle, the product satisfies the Montgomery reducer input bound.

The call to `__fqmul` is discharged by `inverse_twiddle_product_bound_call_ph`. The post-call store reasoning is handled by `inverse_butterfly_step_bound_by`. The array update shape of the right-hand specification differs slightly from the extracted code order, so `inverse_spec_updateE` proves the extensional equality between the generated store expression and the compact mathematical update:

$$(p[j \leftarrow p[j] + p[j+\ell]])[j+\ell \leftarrow \zeta(p[j] - p[j+\ell])].$$

7.4 Final Scaling Loop

After the eight inverse butterfly layers, the coefficients have a uniform 24-bit bound. The final loop applies `__fqmul` at each index with `jzetask_inv[255]`. This loop has its own representation and bound invariant:

$$\text{poly_repr}(rp_1, \text{final_mont_array}(r_2, j_2)) \wedge \text{barray256_bound_by}(rp_1, \text{final_bound}(j_1)).$$

The ghost array

$$\text{final_mont_array}(p, j)[i] = \begin{cases} p[i] \cdot R, & 0 \leq i < j, \\ p[i], & \text{otherwise} \end{cases}$$

records that the prefix already processed by the final loop has been Montgomery-scaled. The corresponding bound function is

$$\text{final_bound}(j, i) = \begin{cases} 16, & 0 \leq i < j, \\ 24, & \text{otherwise.} \end{cases}$$

Thus processed entries are known to be fresh 16-bit Montgomery multiplication outputs, while unprocessed entries still carry the 24-bit bound inherited from the inverse butterfly layers.

The key final-loop lemmas are:

- **final_mont_array_start**: before the loop, the ghost array is the unscaled right-hand result;
- **final_mont_array_setE**: one iteration advances the Montgomery-scaled prefix by one entry;
- **final_mont_array_exit**: after $j = 256$, the ghost array equals `NTT_Fq.array256_mont p`;
- **inverse_final_product_bound_call_ph**: the `__fqmul` call at index 255 computes $\text{coeff} \cdot \text{scale255} \cdot R$ and returns a 16-bit word;
- **inverse_final_product_bound_from_inv**: every final-loop product satisfies the signed Montgomery reducer range because the scale entry is 16-bit and the current unprocessed coefficient is at most 24-bit.

7.5 Completion of the Inverse Proof

The inverse theorem is organized as two pRHL loops after `proc`; `wp`. First, the proof discharges the final scaling loop because it appears syntactically last and is therefore encountered first by weakest precondition reasoning. Then it proves the nested inverse butterfly loops.

At the exit of the inverse butterfly loops, the invariant gives

$$\text{poly_repr}(rp_1, r_2) \wedge \text{barray256_bound_by}(rp_1, \lambda i.24).$$

This state is converted into the precondition of the final-loop invariant by **final_mont_array_start** and **final_bound_start**. At the exit of the final loop, $j = 256$; **final_mont_array_exit** converts the representation target to `NTT_Fq.array256_mont res{2}`, and **final_bound** gives the uniform 16-bit postcondition.

8 Machine-Word Overflow Argument

The overflow argument is not a separate after-the-fact side condition. It is threaded through all pRHL invariants.

The core facts are:

1. All input coefficients begin with a 16-bit bound.
2. Every butterfly layer can increase updated coefficient bounds by at most one bit, as proved by **forward_butterfly_bounds** and **inverse_butterfly_bounds**.

3. There are eight butterfly layers, so a coefficient entering a Montgomery multiplication is never worse than 24-bit.
4. All twiddle-table entries used in multiplication are 16-bit: `jzetas_bound16` and `jzetas_inv_bound16`.
5. A 16-bit twiddle times a 24-bit coefficient satisfies the reducer precondition by `fqmul_product_bound_16_24`.
6. Every result of `__fqmul` is reduced back to a 16-bit word.

This is why the forward theorem ends with a 24-bit bound, while the inverse theorem ends with a 16-bit bound on the Montgomery-scaled output: the inverse has the same eight-layer growth to 24 bits, followed by a final multiplication that reduces every entry back to 16 bits.

9 Proof Engineering Pattern

The proof uses a consistent engineering pattern throughout:

1. Define a small stage-indexing model for the loop counter values.
2. Define a pointwise bound function that distinguishes completed, current, and untouched coefficients.
3. Prove arithmetic transition lemmas for that bound function.
4. Verify the extracted Montgomery multiplication call with a dedicated **phoare** lemma.
5. Verify the subsequent stores by combining field representation lemmas with machine-word add/subtract bounds.
6. Close each loop by converting the inner invariant to the middle invariant, then the middle invariant to the next outer stage invariant.

This structure keeps the pRHL proof direct: the left and right programs are related at the actual loop level, without replacing the extracted code by a separate hand-written model.

10 Checked Artifacts

The relevant checked artifacts are:

- `RefJasminNTT.ec`, containing the pRHL refinements and the supporting representation and bound lemmas;
- `NTT_Fq.ec`, containing the imperative NTT procedures, table definitions, table bridges, and base representation relation;
- `extract/Hpoly_extract.ec`, containing the extracted Jasmin procedures and concrete twiddle tables.

The proof was checked with:

```
easycrypt compile RefJasminNTT.ec -I .
easycrypt compile NTT_Fq.ec -I .
```

and the proof-marker scan

```
rg -n "admit|abort|axiom" RefJasminNTT.ec NTT_Fq.ec
```

reported no proof holes in `RefJasminNTT.ec`. The only reported matches were the word “axiom” in an explanatory comment in `NTT_Fq.ec`.

11 Summary

`RefJasminNTT.ec` proves the direct loop-level extraction refinements for both NTT cores. The forward proof establishes:

$$\text{Hpoly_extract.M._poly_ntt} \sim \text{NTT_Fq.NTT.ntt}$$

under a 16-bit input representation relation, producing a 24-bit represented output. The inverse proof establishes:

$$\text{Hpoly_extract.M._poly_invntt} \sim \text{NTT_Fq.NTT.invntt}$$

under the analogous inverse-table precondition, producing a 16-bit represented output for `array256_mont` of the right-hand result.

The main technical content is the pointwise machine-word bound infrastructure. It rules out overflow locally at every butterfly and at every Montgomery multiplication call, while simultaneously preserving the field-level representation relation needed for the pRHL simulation.